1.Introducción a la Programación

1.1. ¿Qué es la programación?

La **programación** es la disciplina encargada de diseñar, escribir y mantener el código fuente que controla el comportamiento de las computadoras. Es un componente esencial de la informática, y permite crear software que resuelve problemas, automatiza tareas y optimiza procesos.

A través de la **programación**, se transforman ideas en soluciones computacionales, empleando lenguajes específicos que permiten a los programadores comunicarse con el hardware. Esta ciencia combina lógica, matemáticas, y creatividad, permitiendo desarrollar desde aplicaciones sencillas como calculadoras hasta sistemas complejos como los que operan autos autónomos o gestionan redes sociales.

La **programación** también es el motor detrás del desarrollo tecnológico moderno, permitiendo la innovación en áreas como la inteligencia artificial, el análisis de grandes volúmenes de datos (big data), y el internet de las cosas (IoT). Aprender a programar no solo otorga la capacidad de crear software, sino que también fomenta habilidades de pensamiento crítico, resolución de problemas, y abstracción, todas ellas esenciales en un mundo cada vez más digital.

A su vez es el proceso de crear instrucciones que le indican a una computadora cómo realizar tareas específicas. Estas instrucciones se escriben en lenguajes de programación, que actúan como puentes entre el pensamiento humano y el funcionamiento de las máquinas. Python es un lenguaje de programación de alto nivel, conocido por su sintaxis clara y legible, lo que lo convierte en una excelente opción para principiantes.

1.2. Programas y algoritmos

Para entender la relación entre ambos primero se necesitaría saber, de manera breve, que es cada uno.

Un **programa** es un conjunto de instrucciones que una computadora sigue para realizar tareas específicas. Estas instrucciones son escritas en un lenguaje de programación, como Python, y traducidas a un formato que la computadora puede entender y ejecutar.

Cada programa tiene un propósito, ya sea resolver un problema, automatizar procesos o simplemente proporcionar una interfaz para interactuar con el hardware. En esencia, un programa es la implementación concreta de un algoritmo, utilizando los recursos del sistema como la memoria, el procesador y los dispositivos de entrada/salida para realizar las operaciones indicadas.

Un **algoritmo** es una secuencia finita y ordenada de pasos lógicos que resuelven un problema o realizan una tarea. Cada paso en un algoritmo describe una acción que debe ejecutarse en un orden específico para alcanzar el resultado deseado. Los algoritmos no están ligados a ningún lenguaje de programación; en su forma más básica, son descripciones abstractas de soluciones que luego se pueden implementar en cualquier lenguaje.

Ejemplo de un Algoritmo (Instrucciones Cotidianas)

Imaginemos que queremos hacer un sándwich. El algoritmo para esta tarea sería algo así:

1. Tomar dos rebanadas de pan.
2. Colocar una rebanada de jamón sobre una de las rebanadas de pan.
3. Poner una rebanada de queso encima del jamón.
4. Añadir condimentos al gusto (mostaza, mayonesa, etc.).
5. Colocar la otra rebanada de pan encima del queso.
6. Cortar el sándwich por la mitad (opcional).
7. Servir en un plato.

Este conjunto de pasos es un algoritmo porque, siguiendo cada instrucción en el orden indicado, siempre se logra el objetivo final: hacer un sándwich. De manera similar, en programación, un algoritmo define los pasos necesarios para resolver un problema, como ordenar una lista de números o buscar información en una base de datos.

Relación entre Programa y Algoritmo

El algoritmo es el corazón de cualquier programa. Primero, se define el algoritmo que resuelve el problema de manera eficiente y luego, se traduce ese algoritmo en un programa utilizando un lenguaje de programación como Python. En otras palabras, un programa es la "traducción" de un algoritmo en una forma que la computadora puede procesar.

1.3. Conceptos fundamentales: entrada, proceso y salida

Todo programa informático se basa en tres componentes esenciales:

- **Entrada**: Datos o información que el programa recibe para procesar.
- **Proceso**: Operaciones que el programa realiza sobre los datos de entrada.
- **Salida**: Resultados generados después del procesamiento.

Por ejemplo, en un programa que calcula la suma de dos números:

- **Entrada**: Los dos números proporcionados por el usuario.

- **Proceso**: La suma de esos dos números.
- **Salida**: El resultado de la suma.

1.4. Lenguajes de programación

Los lenguajes de programación son herramientas que permiten a los desarrolladores comunicarse con las computadoras. Se dividen en varias categorías:

1.4.1. Lenguaje máquina y ensamblador

- **Lenguaje máquina**: Es el lenguaje más básico, compuesto por códigos binarios (0s y 1s) que la computadora entiende directamente. Es difícil de leer y escribir para los humanos.
- **Lenguaje ensamblador**: Es una representación más legible del lenguaje máquina, utilizando abreviaturas y símbolos. Aunque es más comprensible que el lenguaje máquina, sigue siendo complejo y específico para cada tipo de procesador.

1.4.2. Lenguajes de alto nivel

Son lenguajes más cercanos al lenguaje humano, diseñados para ser fáciles de leer y escribir. Permiten a los programadores escribir instrucciones sin preocuparse por los detalles del hardware subyacente. Python es un ejemplo destacado de un lenguaje de alto nivel.

1.4.3. Lenguajes compilados vs. interpretados

- **Lenguajes compilados**: El código fuente se traduce completamente a lenguaje máquina antes de ejecutarse, mediante un programa llamado compilador. Ejemplos incluyen C y C++.
- **Lenguajes interpretados**: El código fuente se traduce y ejecuta línea por línea en tiempo real, utilizando un intérprete. Python es un lenguaje interpretado, lo que facilita la prueba y depuración del código.

1.4.4. Proceso de compilación

El proceso de compilación es el conjunto de pasos que transforman el código fuente en un programa ejecutable. Este proceso implica varias fases:

- Escritura del código: El programador escribe el código fuente en un lenguaje de programación.
- Compilación: El compilador convierte el código fuente en código objeto (lenguaje de máquina) específico para el hardware.
- Ensamblaje: El código objeto se convierte en un formato que el procesador puede entender.

- Enlazado: Las diferentes partes del programa, así como las bibliotecas externas, se combinan para formar el ejecutable final.
- Importancia del Compilador y Enlazador: El compilador es clave para traducir el código a un lenguaje que la computadora entienda, mientras que el enlazador se encarga de juntar todas las partes del programa, incluyendo las funciones y bibliotecas, para generar el archivo ejecutable.

1.5. Lógica y pensamiento computacional

La **lógica** es la base del pensamiento computacional, y se refiere a la capacidad de estructurar soluciones a problemas complejos de manera clara y secuencial. En la programación, esto significa descomponer un problema grande en sub-problemas más pequeños, resolverlos uno por uno y combinarlos para alcanzar la solución final.

- Descomposición de Problemas: En la programación, los problemas grandes se resuelven dividiéndolos en partes manejables, lo que facilita la creación de soluciones eficientes. Este proceso, llamado descomposición, es esencial para escribir programas bien estructurados.
- Importancia de la Lógica: La lógica es fundamental en la programación, ya que permite tomar decisiones y controlar el flujo del programa mediante estructuras condicionales y ciclos.

Ejemplo de Lógica y Pensamiento Computacional: Preparar un Sándwich

Problema: Quieres hacerte un sándwich, pero necesitas asegurarte de que tienes todos los ingredientes necesarios y seguir una secuencia lógica de pasos.

- Descomposición del Problema:
  - Paso 1: Verificar si tienes pan.
  - Paso 2: Verificar si tienes los ingredientes para el relleno (queso, jamón, etc.).
  - Paso 3: Si tienes pan y relleno, entonces puedes hacer el sándwich.
  - Paso 4: Si falta algún ingrediente, decide si irás a comprarlo o hacer otra cosa para comer.
- Decisiones Lógicas (Condicionales):
  - Condicional 1: ¿Tienes pan?
    - Si NO, decide si ir a comprar pan o hacer otra cosa para comer.
    - Si SÍ, continúa al siguiente paso.
  - Condicional 2: ¿Tienes los ingredientes del relleno?
    - Si NO, decide si comprar los ingredientes faltantes o hacer un sándwich con lo que tengas.
    - Si SÍ, procede a hacer el sándwich.

Este ejemplo muestra cómo la lógica y el pensamiento computacional te ayudan a descomponer el problema en partes más pequeñas (pan, ingredientes) y tomar decisiones basadas en las condiciones (si tienes o no los ingredientes necesarios).

1.6. Diagramas de flujo y pseudocódigo

Antes de escribir código, es útil planificar la lógica del programa:

- **Diagramas de flujo**: Representaciones gráficas que muestran el flujo de control de un algoritmo, utilizando símbolos estandarizados para diferentes tipos de acciones y decisiones.
- **Pseudocódigo**: Descripción textual de un algoritmo que combina elementos del lenguaje natural con estructuras de programación. No sigue la sintaxis de un lenguaje de programación específico, pero ayuda a visualizar la lógica antes de la implementación.

2. Paradigmas de programación

2.1. Introducción a los paradigmas de programación

Un **paradigma de programación** es un enfoque o estilo de escribir código que define cómo estructurar y organizar un programa para resolver problemas. A lo largo de la historia de la informática, han surgido distintos paradigmas con diferentes formas de pensar y modelar la solución de un problema en código.

Algunos paradigmas se enfocan en la manera en que se ejecutan las instrucciones (**imperativa vs. declarativa**), mientras que otros definen cómo se organiza la estructura del código (**estructurada, orientada a objetos, funcional, lógica, etc.**).

Un mismo lenguaje de programación puede admitir más de un paradigma. Por ejemplo, **Python** permite escribir código en un estilo **imperativo, estructurado, procedural, orientado a objetos y funcional**, dependiendo de cómo se use.

En este capítulo nos enfocaremos en los paradigmas más utilizados, con especial énfasis en la **programación estructurada**

2.1.2. Programación Imperativa

La **programación imperativa** es el paradigma más antiguo y se basa en dar instrucciones secuenciales al computador para modificar el estado del programa. Se le llama "imperativa" porque indica cómo deben ejecutarse las operaciones paso a paso.

Características principales:

- Uso de variables que almacenan valores y pueden modificarse.
- Uso de estructuras de control (**if**, **while**, **for**, etc.) para controlar el flujo de ejecución.
- El código se ejecuta de forma secuencial, de arriba hacia abajo.

2.1.3. Programación Estructurada (subtipo de imperativa)

La **programación estructurada** surgió en los años 70 como una evolución de la programación imperativa para mejorar la claridad y mantenimiento del código.

Principios básicos:

1. **Secuencia:** Instrucciones ejecutadas en orden.
2. **Selección:** Permite tomar decisiones (**if, if-else**).
3. **Iteración:** Repite un bloque de código (**while, for**).

Se eliminó el uso del **goto**, ya que producía código difícil de leer y mantener ("código espagueti").

En el caso de Python, **no admite goto de forma nativa**. Sin embargo, existe un módulo llamado **goto** en la librería **goto-statement** que permite simularlo en Python, aunque **su uso es altamente desaconsejado, tanto en Python como en cualquier otro lenguaje,** porque va en contra de las buenas prácticas de programación estructurada.

2.1.4. Programación Procedural (subtipo de estructurada)

También conocida como **programación basada en procedimientos**, es una extensión de la programación estructurada donde el código se organiza en **funciones** o **procedimientos** reutilizables.

Características:

- Divide el código en funciones que encapsulan bloques de lógica.
- Permite reutilización y mejor mantenimiento.
- Usa llamadas a funciones para ejecutar procedimientos específicos.

2.1.5. Programación Orientada a Objetos (POO)

La **POO** es un paradigma que modela los problemas en términos de **objetos** y **clases**. Es útil para desarrollar software modular y reutilizable.

Principios de la POO:

1. **Encapsulamiento:** Agrupa datos y funciones dentro de una clase.
2. **Herencia:** Permite que una clase reutilice y extienda el comportamiento de otra.
3. **Polimorfismo:** Permite usar el mismo método en diferentes contextos, entre otras cosas.

2.1.6. Programación Declarativa

A diferencia de la programación imperativa, que indica **cómo** hacer algo, la programación declarativa se enfoca en **qué** se quiere lograr, sin especificar paso a paso cómo se hace. Un ejemplo de este paradigma es **SQL**, que se usa para consultar bases de datos sin especificar cómo recuperarlas